

Observer Pattern in C#

Klassische Umsetzung vs. Umsetzung mit .NET-Events — Aktienkurs-Simulation

Inhalt

1. Was ist das Observer Pattern?
2. Aufbau & Ablauf
3. Variante 1: Klassisches Observer Pattern
4. Variante 2: Observer Pattern mit Events
5. Vergleich der beiden Varianten
6. Beispiel-Ausgabe

1. Was ist das Observer Pattern?

Das **Observer Pattern** ist ein Verhaltensmuster (Behavioral Pattern) aus dem klassischen "Gang of Four"-Katalog. Es beschreibt eine **1:n-Beziehung** zwischen einem Objekt (**Subject**) und mehreren abhängigen Objekten (**Observern**). Ändert sich der Zustand des Subjects, werden automatisch alle registrierten Observer benachrichtigt – ohne dass das Subject wissen muss, was die Observer konkret mit dieser Information machen.

Kernidee: Lose Kopplung. Das Subject kennt nur, dass es Observer gibt (über eine gemeinsame Schnittstelle bzw. Signatur) – nicht aber deren konkrete Klasse oder Implementierung.

In diesem Projekt wird das Pattern anhand einer **Aktienkurs-Simulation** demonstriert: Mehrere Aktien (`AAPL` , `MSFT` , `TSLA`) ändern in zufälligen Schritten ihren Kurs. Ein **Ticker** (Kursanzeige) meldet jede Änderung, und mehrere **Investoren** haben jeweils gezielt bestimmte Aktien „abonniert“ und werden nur über deren Kursänderungen informiert.

2. Aufbau & Ablauf

1. Ein Investor/Ticker registriert sich beim Subject (Aktie) → `Attach()` bzw. `+=`
2. Die Aktie ändert ihren Kurs (zufällig, über `Fluctuate()`)
3. Die Aktie benachrichtigt **alle** registrierten Observer in der Reihenfolge ihrer Anmeldung
4. Jeder Observer reagiert individuell (z. B. Konsolen-Ausgabe)
5. Ein Observer kann sich jederzeit wieder abmelden → `Detach()` bzw. `-=`

3. Variante 1: Klassisches Observer Pattern

Hier wird das Pattern manuell über zwei Schnittstellen umgesetzt: `ISubject` und `IObserver`. Das Subject (`Stock`) verwaltet die Liste der Observer selbst in einer `List<IObserver>` und ruft bei jeder Änderung `Notify()` auf.

Interfaces/IObserver.cs

```
public interface IObserver
{
    void Update(Stock stock, decimal price);
}
```

Interfaces/ISubject.cs

```
public interface ISubject
{
    void Attach(IObserver observer);
    void Detach(IObserver observer);
    void Notify();
}
```

Stock.cs – das Subject

```
public class Stock : ISubject
{
    private readonly List<IObserver> _observers = new();
    private decimal _price;
    public string Symbol { get; }

    public Stock(string symbol, decimal startPrice)
    {
        Symbol = symbol;
        _price = startPrice;
    }

    public void Attach(IObserver observer) => _observers.Add(observer);
    public void Detach(IObserver observer) => _observers.Remove(observer);

    public void Notify()
    {
        foreach (var observer in _observers)
        {
            observer.Update(this, _price);
        }
    }

    public void SetPrice(decimal price)
    {
        _price = price;
        Notify();
    }

    public void Fluctuate(Random random)
    {
        int change = random.Next(-5, 6); // -5 bis +5
        decimal newPrice = _price + change;

        if (newPrice < 1)
        {
            newPrice = 1; // Preis soll nicht negativ/null werden
        }

        SetPrice(newPrice);
    }
}
```

TickerDisplay.cs – Observer

```
public class TickerDisplay : IObserver
{
    public void Update(Stock stock, decimal price)
    {
        Console.WriteLine($"[Ticker] {stock.Symbol} Kurs aktualisiert auf {price}€");
    }
}
```

Investor.cs – Observer

```
public class Investor : IObserver
{
    private string Name;
```

```

public Investor(string name)
{
    Name = name;
}

public void Update(Stock stock, decimal price)
{
    Console.WriteLine($"{Name} wurde informiert: {stock.Symbol} jetzt bei {price:C}");
}
}

```

Program.cs – Aufbau der Simulation

```

Console.OutputEncoding = Encoding.UTF8;

var random = new Random();

var apple = new Stock("AAPL", 150.00m);
var microsoft = new Stock("MSFT", 310.00m);
var tesla = new Stock("TSLA", 230.00m);

var ticker = new TickerDisplay();

var investorA = new Investor("Nowak");
var investorB = new Investor("Lisa");
var investorC = new Investor("Hofihasi");

// ticker "abonnieren", sodass Meldung ueber Kursaenderung ausgegeben wird
apple.Attach(ticker);
microsoft.Attach(ticker);
tesla.Attach(ticker);

// Investoren, welche apple "abonniert" haben
apple.Attach(investorA);
apple.Attach(investorB);

// Investoren, welche microsoft "abonniert" haben
microsoft.Attach(investorA);
microsoft.Attach(investorC);

// Investoren, welche tesla "abonniert" haben
tesla.Attach(investorB);
tesla.Attach(investorC);

var stocks = new List<Stock> { apple, microsoft, tesla };

for (int i = 0; i < 10; i++)
{
    Console.WriteLine($"----- Runde {i + 1} -----");
    foreach (var stock in stocks)
    {
        stock.Fluctuate(random);
        Console.WriteLine();
    }
    Thread.Sleep(2000);
}

```

4. Variante 2: Observer Pattern mit Events

Diese Variante nutzt C#s eingebautes Event-System (`event` + `Delegates`). Statt einer eigenen `IObserver` -Schnittstelle und manueller Listenverwaltung übernimmt der Compiler das An- und Abmelden über `+=` und `-=` . Zusätzlich folgt sie der .NET-Konvention: Ein Event liefert (`object? sender, EventArgs e`) , wobei `EventArgs` von `EventArgs` erbt.

Events/PriceChangedEventArgs.cs

```
public class PriceChangedEventArgs : EventArgs
{
    public decimal NewPrice { get; }

    public PriceChangedEventArgs(decimal newPrice)
    {
        NewPrice = newPrice;
    }
}
```

Events/StockEvent.cs – das Subject

```
public class StockEvent
{
    public string Symbol { get; }
    private decimal _price;

    public event EventHandler<PriceChangedEventArgs>? PriceChanged;

    public StockEvent(string symbol, decimal startPrice)
    {
        Symbol = symbol;
        _price = startPrice;
    }

    public void SetPrice(decimal price)
    {
        _price = price;
        OnPriceChanged(new PriceChangedEventArgs(price));
    }

    public void Fluctuate(Random random)
    {
        int change = random.Next(-5, 6);
        decimal newPrice = _price + change;

        if (newPrice < 1)
        {
            newPrice = 1; // Preis soll nicht negativ/null werden
        }

        SetPrice(newPrice);
    }

    // Konvention: geschuetzte "On<EventName>"-Methode zum Ausloesen
    protected virtual void OnPriceChanged(PriceChangedEventArgs e)
    {
        PriceChanged?.Invoke(this, e);
    }
}
```

Events/TickerDisplayEvent.cs – Observer

```
public class TickerDisplayEvent
{
    public void OnPriceChanged(object? sender, PriceChangedEventArgs e)
    {
        var stock = sender as StockEvent;
        Console.WriteLine($"[Ticker] {stock?.Symbol} Kurs aktualisiert auf {e.NewPrice}€");
    }
}
```

Events/InvestorEvent.cs – Observer

```
public class InvestorEvent
{
    public string Name { get; }

    public InvestorEvent(string name)
    {
        Name = name;
    }

    public void OnPriceChanged(object? sender, PriceChangedEventArgs e)
    {
        var stock = sender as StockEvent;
        Console.WriteLine($"[{Name}] wurde informiert: {stock?.Symbol} jetzt bei {e.NewPrice}€");
    }
}
```

```
}
```

Program.cs – Aufbau der Simulation

```
Console.OutputEncoding = Encoding.UTF8;

var random = new Random();

var apple = new StockEvent("AAPL", 150.00m);
var microsoft = new StockEvent("MSFT", 310.00m);
var tesla = new StockEvent("TSLA", 230.00m);

var ticker = new TickerDisplayEvent();

var investorA = new InvestorEvent("Nowak");
var investorB = new InvestorEvent("Lisa");
var investorC = new InvestorEvent("Hofihasi");

// ticker "abonnieren", sodass Meldung ueber Kursaenderung ausgegeben wird
apple.PriceChanged += ticker.OnPriceChanged;
microsoft.PriceChanged += ticker.OnPriceChanged;
tesla.PriceChanged += ticker.OnPriceChanged;

// Investoren, welche apple "abonniert" haben
apple.PriceChanged += investorA.OnPriceChanged;
apple.PriceChanged += investorB.OnPriceChanged;

// Investoren, welche microsoft "abonniert" haben
microsoft.PriceChanged += investorA.OnPriceChanged;
microsoft.PriceChanged += investorC.OnPriceChanged;

// Investoren, welche tesla "abonniert" haben
tesla.PriceChanged += investorB.OnPriceChanged;
tesla.PriceChanged += investorC.OnPriceChanged;

var stocks = new List<StockEvent> { apple, microsoft, tesla };

for (int i = 0; i < 10; i++)
{
    Console.WriteLine($"----- Runde {i + 1} -----");
    foreach (var stock in stocks)
    {
        stock.Fluctuate(random);
        Console.WriteLine();
    }
    Thread.Sleep(2000);
}
```

5. Vergleich der beiden Varianten

Aspekt	Klassisch (Interfaces)	Mit Events
Registrierung	Attach() / Detach()	+= / -=
Benötigte Typen	ISubject , IObserver	event -Feld, EventArgs -Klasse
Boilerplate	Mehr (eigene Interfaces, Listen-Verwaltung)	Wenig (Sprachfeature übernimmt das)
Kopplung an Observer	Observer muss IObserver implementieren	Beliebige passende Methode reicht
Wer wird informiert?	Reihenfolge = Reihenfolge von Attach()	Reihenfolge = Reihenfolge von +=
Idiomatisch in C#?	Eher für GoF-Demonstration	Ja – Standard in .NET (Buttons, UI, etc.)

6. Beispiel-Ausgabe

Beide Varianten liefern – bei gleicher Konfiguration – dieselbe Art von Ausgabe. Der Ticker meldet sich zuerst (da er als Erstes registriert wird), danach folgen die jeweils betroffenen Investoren:

Konsolen-Ausgabe (Ausschnitt, Werte durch Zufall bei jedem Durchlauf anders)

```
----- Runde 1 -----
[Ticker] AAPL Kurs aktualisiert auf 153€
Nowak wurde informiert: AAPL jetzt bei 153,00 €
Lisa wurde informiert: AAPL jetzt bei 153,00 €

[Ticker] MSFT Kurs aktualisiert auf 307€
Nowak wurde informiert: MSFT jetzt bei 307,00 €
Hofihasi wurde informiert: MSFT jetzt bei 307,00 €

[Ticker] TSLA Kurs aktualisiert auf 228€
Lisa wurde informiert: TSLA jetzt bei 228,00 €
Hofihasi wurde informiert: TSLA jetzt bei 228,00 €

----- Runde 2 -----
...
```

Zu beachten: Nowak und Lisa sind bei AAPL registriert, Nowak und Hofihasi bei MSFT , Lisa und Hofihasi bei TSLA . Jeder Investor wird also **ausschließlich** über die Aktien informiert, die er zuvor abonniert hat – genau das ist der Kern des Observer Patterns: gezielte, lose gekoppelte Benachrichtigung statt globaler Broadcasts an alle.